Pip Architecture — F5: the agent loop (plan)

FOLIOS · FOLIOSHQ.COM

Status: plan → build, on branch `claude/f5-agent-loop-gtbcok`. NOT deployed; Chris fast-forwards `main` after he tests. This is the last open item in the X6 FORM sequence (F1 shared builder → F2 persist → F3 event-driven → **F5 agent loop** → F6 pgvector recall).

One-line goal: give Pip's chat a real tool-use loop — model emits a tool call → server executes it → returns the result → model keeps reasoning → final answer — so Pip can do multi-step work in one turn ("find the stalled project, then draft the chase note") instead of one isolated action.

The #1 rule: do not regress today's chat path

Today's chat is **single-shot**: the model emits text + tool_use blocks, the server streams the text and forwards the tool_use blocks to the client, and the **client** executes them (frictionless ones fire immediately; the rest go through a confirm card). The tool *result* never goes back to the model. (`api/pip.js` streaming branch → `src/lib/pipStream.js` → `PipView.send` → `planToolCalls` / `executeTool`.)

That single-shot behavior is the **floor**. The loop is additive and degrades to it cleanly:

- The loop activates **only** when the model emits a *read tool* (a new, server-executed, read-only tool family). If the model emits no read tool — which is every chat turn today — the loop runs exactly one model call and behaves byte-identically to today.
- A kill switch (`PIP_AGENT_LOOP=off`) removes read tools from the tools array entirely, so the model *can't* call them → guaranteed single-shot.

- On any loop error or cap, we land on a clean single text answer — never a broken turn.

Why the loop needs NEW read tools (the key architectural call)

Pip's *existing* tools (`src/lib/pipTools.js`) cannot be the thing the loop iterates on, for two reasons:

1. **They execute client-side.** `open_*` / `navigate` are pure UI actions with no meaningful "result." The DB-write tools (`create_open_item` , `log_meeting` , …) run against the user's Supabase session via React hooks in `PipView` . The server has no way to produce a `tool_result` for them.
2. **Executing writes server-side would regress the confirm-card safety model.** Today a write goes through a confirm card (`needsConfirm` → `category: "confirm"`). Running it server-side mid-loop would silently commit it. Unacceptable.

So the existing **action tools stay exactly as today: client-executed and *terminal* within a chat turn.** The loop iterates on a small, new family of **read-only, server-executed tools** that let Pip *gather* mid-turn. A turn is naturally either "gathering" (read tools → loop continues) or "acting" (an action tool → terminate, hand to client) — matching how Pip should work: look first, then act.

This is the honest reading of "expose existing tools": the existing tools' behavior is preserved verbatim; we add the minimal read surface a real loop requires.

The read tools (v1)

All three are read-only, scoped to the caller via the JWT-attached Supabase client (RLS does the rest), and return the user's **own notebook data** — which the Data Line Rule explicitly permits Pip to read (the rule governs Pip *soliciting* and *retaining* business data, not reading Chris's own verbatim notes). None solicit or persist numbers.

Tool	Input	What it returns	Renderer
`lookup_account`	`account_name`	Deep current detail for ONE account: recent meetings (with notes), open items, contacts, active projects. For accounts not in the trimmed context, or deeper than the summary.	**`accountContext.renderAccount{ {surface:"chat"})`** — the F1 sha PARITY by construction.
`find_open_work`	`filter ∈ overdue | stalled | waiting_on_them | due_soon | all`	Concise cross-portfolio rows (account · what · status · age · who-has-ball). Serves "find the stalled project / what's slipping / who has the ball."	compact line list (query result, nc context — no parity surface)
`search_notes`	`query`	Matching snippets from the user's meeting	compact snippet list

		notes + Pip summaries across all accounts (account · date · snippet). Serves "where/when was X discussed."	

Read tools **never cross to the client** — they're a server-internal working set. The client only ever receives action tools (it has no executor for read tools). This is a hard invariant in the forwarding code.

WHERE the loop lives

- **`src/lib/pipAgentTools.js`** (NEW, pure — no Supabase/React/fetch, unit-tested): the read-tool definitions (`PIP_READ_TOOLS`), `isReadTool` , `partitionToolUses(content)` → `{readTools, actionTools, text}` , and `decideLoopStep({partition, step, maxSteps})` → `"continue" | "terminate" | "force_final"` . The risky control logic is pure and tested here.
- **`api/_pipAgentLoop.js`** (NEW, server I/O — receives the Anthropic `client` as a param, so it does NOT import the SDK and stays clear of Guard 3): `executeReadTool(name, input, ctx)` (Supabase queries + `accountContext` render) and `runAgentChat({...})` (drives the loop using the pure helpers).
- **`api/pip.js`** (chat streaming + buffered branches): when chat mode + loop enabled, route through `runAgentChat` ; otherwise the existing inline single call, untouched. `tools` becomes `PIP_TOOLS.concat(PIP_READ_TOOLS)` only when the loop is on.

`.js` extensions on all relative imports (CLAUDE.md API Module Import Rule). `api/_pipAgentLoop.js` is a helper (like `_pipUsage.js`), not a handler, so it needs no entry in `test-api-imports.js`, but it's import-tested transitively when `api/pip.js` loads.

The loop (mechanics)

```
tools = PIP_TOOLS + PIP_READ_TOOLS         // byte-stable across all
requests & iterations
system = [PIP_STATIC_SYSTEM (cache_control), dynamic tail]   //
identical object reused every iteration
messages = [ ...chat history ]

for step in 0 .. maxSteps-1:
    forceFinal = (step === maxSteps-1)
    params = { model, max_tokens, system, tools, messages,
               tool_choice: forceFinal ? {type:"none"} : undefined }
// none keeps tools+system cache valid
    stream  = client.messages.stream(params)
    stream.on("text", delta => sseDelta(delta))      // stream EVERY
turn's text live
    final   = await stream.finalMessage()
    logPipUsage(... final.usage ...)                 // log EACH model
call → spend tile sees full cost

    { readTools, actionTools } = partitionToolUses(final.content)

    // Continue ONLY when the turn is pure gathering: read tools
present AND no action tools.
    if (readTools.length === 0 || actionTools.length > 0 ||
forceFinal):
        return { fullText, actionToolCalls: actionTools }   //
TERMINAL

    messages.push({ role:"assistant", content: final.content })
// includes the read tool_use blocks
    results = await Promise.all(readTools.map(executeReadTool))
// each → tool_result block (is_error on failure)
    messages.push({ role:"user", content: results })
// every tool_use answered → API satisfied
```

Then the handler emits the existing SSE shape: `delta` events already streamed, one `tool_use` event per **action** tool (read tools filtered out), and `done` with the accumulated `content` + action `tool_calls`. **Zero client changes** — `pipStream.js` and `PipView` see the same protocol they see today.

Why "terminate when any action tool appears"

The API requires a `tool_result` for *every* `tool_use` in the prior assistant turn. If a turn mixed a read tool and an action tool, continuing the loop would leave the action tool's `tool_use` unanswered (we can't run it server-side). So a turn with any action tool is **terminal** — all action tools go to the client, and any read tool in that same terminal turn is simply not executed (rare; the model already has enough to act). The loop continues only on pure read-only turns. Clean, and no dangling `tool_use`.

HARD cost guard

- **Step cap:** `maxSteps` = **4** model calls (≤ 3 read round-trips), env-overridable via `PIP_AGENT_MAX_STEPS`. On the final allowed step, `tool_choice: {type:"none"}` forces a text answer — graceful landing, never a half-finished loop.
- **Per-call ceiling:** `max_tokens` stays 900 (chat config) per call.
- **Full visibility:** `logPipUsage` fires on **every** model call, so the Settings spend tile and `folio_pip_usage` reflect true loop cost (no invisible spend — the failure class Batch 1 fixed).

Worst-case cost per chat turn (Sonnet $3/M in, $15/M out; chat `max_tokens` 900):

- A turn that *doesn't* loop (the overwhelming majority — same as today): **1 call.**
- Absolute worst case: 4 calls. Per call ≈ static system (~~1.4k tok, **cache-read** at 0.1× after call 1) + context tail + growing tool_result blocks~~ (uncached, est. 4–8k tok) + ≤900 output. ≈ `(6k×0.3 + 900×1.5)/1000` ≈ **$0.0045/M-**

scaled ≈ ~$0.003/call → ~$0.012 worst case for a fully-looped turn. Even pessimistically padding context to 12k input/call, worst case stays **under ~$0.03/turn**, and only on turns where Pip genuinely needed 3 lookups. The step cap makes runaway impossible; the daily spend cap (`overDailySpendCap`) remains the portfolio-level backstop.

Streaming

Chat streams today. The loop preserves it: **text deltas stream live on every iteration** (the brief "let me check that project…" preamble of a gathering turn, then the final answer), accumulating into `done.content`. Tool round-trips happen entirely server-side — the client never sees a read tool or a `tool_result`. The SSE protocol (`meta` / `delta` / `tool_use` / `done` / `error`) is unchanged, so `pipStream.js` needs no edits.

Prompt cache discipline

- `tools` (= `PIP_TOOLS` + `PIP_READ_TOOLS`) is byte-stable across all requests and every loop iteration → the tools cache prefix holds.
- The same `system` blocks object (static block carries `cache_control:ephemeral`) is reused on every iteration — never rebuilt mid-loop → system cache holds from iteration 2 onward.
- Only `messages` grows (appended assistant `tool_use` + user `tool_result`); the cached `tools` + `system` prefix is read, not rewritten. `tool_choice:none` on the forced-final call invalidates only the messages tier, not tools/system (per the caching invalidation hierarchy).

Fallback (current behavior is the guaranteed floor)

- **Loop disabled** (`PIP_AGENT_LOOP=off`): read tools are not added to the tools array; the model can't call them; the existing single inline call runs → byte-identical to today.
- **Read-tool execution error:** returns a `tool_result` with `is_error:true` + a short message → the model recovers or answers without it. Never throws the turn.
- **Cap reached:** the final step runs with `tool_choice:none` → a clean text answer.
- **Stream/model error mid-loop:** caught; if text was already produced, emit `done` with it; else emit the existing `error` event. Same as today's catch.

Parity (F1)

`lookup_account` renders the deep account read through **`accountContext.renderAccountContext(bundle, {surface:"chat"})`** — the one shared builder F1 introduced — so a field added to that builder reaches the loop for free, exactly as it reaches chat/brief/summarize/operator. No new bespoke account renderer. `find_open_work` / `search_notes` return query rows, not "what Pip knows about an account," so they don't touch the parity surface.

Data Line Rule

Read tools read Chris's own verbatim notebook — explicitly allowed. They **never solicit** numbers (tool descriptions ask for account names / topics / status filters, never

figures) and **never retain** anything (reads write nothing). Documented in `ai-governance.md` (how the agent loop is bounded) + `data-handling.md`.

Tests (drift + behavior locks; node-only vitest env, so logic is pure-tested)

New `src/lib/pipAgentTools.test.js`:

- `PIP_READ_TOOLS` shape (name/description/input_schema present; all read-only).
- `isReadTool` correctly classifies read vs action tool names.
- `partitionToolUses` splits a mixed content array into read/action/text.
- `decideLoopStep`: pure read-only turn → `continue`; turn with an action tool → `terminate`; no tools → `terminate`; last step → `force_final`.
- Invariant: an action-tool-only turn never continues (no-regression lock).

Existing 330 tests must stay green. Gates every commit: `vite build` · `vitest` · `node scripts/check-guards.js` · `node scripts/test-api-imports.js`.

Out of scope (deliberately)

- Server-executing write/action tools (would regress the confirm card).
- Client changes to `pipStream.js` / `PipView` (the loop is server-internal).
- F6 pgvector recall (separate item; `search_notes` here is plain ilike, not semantic).
- Looping the non-chat modes (summary/brief/email/action) — those keep today's behavior exactly.